CYBERSECURITY AVANZATA · LEZIONE 2 DI 8

# XSS, CSRF e Content Security Policy

Quando il browser della vittima lavora per l'attaccante: tre tipi di XSS, le richieste falsificate e le difese che il server può imporre al browser.

5G TPSI

XSS riflesso, memorizzato e DOM, escaping, CSP, CSRF, SameSite

PER COMINCIARE

# Quale sito ti fideresti di più a usare da loggato?

SONDAGGIO

- **A** Uno con HTTPS e il lucchetto
- **B** Uno famoso, con milioni di utenti
- **C** Uno che non mostra pubblicità di terzi
- **D** Nessuno: dipende da come è scritto il codice

IL PROBLEMA

# XSS: il dato diventa codice nella pagina

### Riflesso

Lo script arriva nella richiesta (per esempio nell'URL) e il server lo rimanda nella pagina. Serve che la vittima apra il link.

### Memorizzato

Lo script viene salvato nel database (un commento, un profilo) e parte per **chiunque** apra la pagina.

### DOM

Il server non c'entra: è il JavaScript della pagina a inserire un dato non fidato con `innerHTML`.

> In tutti e tre i casi lo script gira **con l'origine del sito**: legge la pagina, invia richieste a nome dell'utente, ruba ciò che non è protetto.

UNA STORIA DEL 2005

# Un milione di amici in meno di un giorno

Ottobre 2005, MySpace. Samy Kamkar, 19 anni, nasconde uno script nel suo profilo. Chi lo visita aggiunge Samy agli amici, scrive "samy is my hero" sul proprio profilo e **copia lo script** anche lì.

In meno di 24 ore oltre un milione di profili infetti. MySpace va offline per ripulire.

### Com'è finita

Indagine federale. Samy patteggia: tre anni di libertà vigilata, lavori socialmente utili e il divieto di usare Internet per un periodo. Oggi è un ricercatore di sicurezza noto.

AL COMPUTER · 15 minuti

# Juice Shop: un XSS nel DOM

1. Avvia Juice Shop in locale come nella lezione 5 e apri **http://localhost:3000**.
2. Nella barra di ricerca scrivi `<b>ciao</b>`: la parola appare in grassetto?
3. Ora cerca: `<iframe src="javascript:alert(`xss`)">`
4. In **xss-cognome.md** spiega perché il grassetto del passo 2 era già un campanello d'allarme.

**Controllo:** compare un popup con **xss** e lo Score Board segna risolta la sfida **DOM XSS**.

QUIZ A TRABOCCHETTO

# Il sito sostituisce "<script>" con una stringa vuota in ogni commento. È protetto dall'XSS?

QUIZ

A Sì: senza il tag script non si esegue JavaScript

B No: ci sono molti altri modi, e il filtro stesso si aggira

C Sì, ma solo se usa anche HTTPS

D No, ma solo per gli utenti di browser vecchi

LA DIFESA PRINCIPALE

# Escaping: il dato resta dato

| Carattere | Diventa |
| --- | --- |
| & | & |
| < | < |
| > | > |
| " | " |
| ' | ' |

- Si applica **in output**, nel punto in cui il dato entra nell'HTML
- Il contesto conta: dentro un attributo, in JavaScript o in un URL servono regole diverse
- I framework (Angular, React, i template JSP con `c:out`) lo fanno da soli: il pericolo è quando lo si spegne
- Nel DOM: `textContent` al posto di `innerHTML`

AL COMPUTER · 30 minuti

# Un XSS riflesso in Java, poi la correzione

```
s.createContext("/cerca", ex -> {
  String q = URLDecoder.decode(ex.getRequestURI()
      .getRawQuery().substring(2), UTF_8);
  String html = "<h1>Risultati per: " + q + "</h1>";
  invia(ex, html);
});
```

1. Completa **Cerca.java** con `HttpServer` sulla porta 8080 e avvialo.
2. Apri `localhost:8080/cerca?q=<script>alert(1)</script>`
3. Scrivi `static String escape(String s)` e usala su `q`. Riprova.

**Controllo:** prima appare il popup. Dopo la correzione la pagina mostra il testo `<script>alert(1)</script>` e nel sorgente (Ctrl+U) leggi `<script>`.

PUZZLE

# Che tipo di XSS è? Abbina

| | |
|---|---|
| **Una recensione con uno script, salvata nel database del negozio** | Basato sul DOM |
| **Un link per e-mail con lo script nel parametro di ricerca** | Riflesso |
| `el.innerHTML = location.hash` | Memorizzato |
| **Il nome utente mostrato con `textContent`** | Nessuno: è sicuro |

IL SECONDO STRATO

# Content Security Policy: il server detta le regole al browser

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'
```

### script-src 'self'

Solo script dai file del sito. Gli script **inline** e gli `onclick=` sono bloccati.

### Nonce

Uno script inline è ammesso solo se porta il numero casuale scritto nell'header di quella risposta.

### Report

Con `-Report-Only` la policy non blocca, ma segnala: per provarla senza rompere il sito.

AL COMPUTER · 15 minuti

# Accendi la CSP e guarda che cosa blocca

1. Torna alla versione **vulnerabile** di Cerca.java (senza escape).
2. Prima di inviare la risposta aggiungi l'header `Content-Security-Policy` con valore `script-src 'self'`.
3. Riapri l'URL con lo script, poi apri DevTools, scheda **Console**.
4. Annota in **xss-cognome.md** il messaggio e perché la CSP da sola non basta.

**Controllo:** il popup non compare e la console dice che il browser si è rifiutato di eseguire uno script inline perché viola la Content Security Policy.

UN'ALTRA TRUFFA DEL BROWSER

# CSRF: il tuo browser firma al posto tuo

Sei loggato su **banca.example**. Apri una pagina di un altro sito che contiene:

```
<form action="https://banca.example/bonifico" method="post">
  <input name="iban" value="IT00…attaccante">
  <input name="importo" value="500">
</form>
<script>document.forms[0].submit()</script>
```

## Che cosa succede

Il browser invia la richiesta **con i tuoi cookie**. Se la banca controlla solo la sessione, il bonifico parte. L'attaccante non legge nulla: gli basta che l'azione avvenga.

QUIZ

# Quale difesa ferma il CSRF del bonifico, ma non un XSS sulla stessa banca?

QUIZ

A Il cookie di sessione con SameSite=Strict

B L'escaping dell'output in tutte le pagine

C Una Content Security Policy senza 'unsafe-inline'

D HTTPS con HSTS

AL COMPUTER · 20 minuti

# Leggi i cookie di un sito vero, senza toccare nulla

1. Accedi con il tuo account scolastico a un servizio che usi già (per esempio Classroom).
2. DevTools, scheda **Application**, voce **Cookies**: guarda le colonne **HttpOnly**, **Secure** e **SameSite**.
3. Fai lo stesso su Juice Shop, dopo il login.
4. In **xss-cognome.md** confronta: quali cookie di sessione sono protetti, e da che cosa?

**Controllo:** nei servizi grandi i cookie di sessione hanno quasi sempre **Secure** e **HttpOnly**. In Juice Shop il token si trova anche in un cookie leggibile dal JavaScript: un XSS potrebbe rubarlo.

UNA LEZIONE DEL 1984

# Di quale codice ti puoi fidare?

> **Non puoi fidarti del codice che non hai scritto tu, completamente.**
>
> **Ken Thompson**, co-autore di Unix, "Reflections on Trusting Trust", discorso per il Premio Turing, 1984

Ogni `<script src>` di un altro sito è codice di altri che gira con i tuoi permessi. La CSP serve anche a decidere **di chi** fidarsi.

OGGI

# Il browser aiuta, l'AI a volte no





SFIDA A SQUADRE

# Apri il lucchetto



- Il codice è l'anno in cui un worm XSS ha aggiunto un milione di amici a un ragazzo di 19 anni.
- Attenzione: avete solo tre tentativi prima dell'indizio.

codice

BIGLIETTO D'USCITA

# Completa prima di uscire

Uno script salvato in un commento e servito a tutti è un XSS _____.

Per inserire testo nel DOM senza rischi si usa _____ al posto di innerHTML.

Il carattere < in HTML si scrive _____.

L'header che decide da dove si caricano gli script è la Content _____.

L'attributo del cookie contro il CSRF è _____.

⏱ 3:00

RIEPILOGO

# Cosa portiamo a casa

- XSS riflesso, memorizzato, DOM
- Liste nere in input: si aggirano sempre
- Escaping in output, textContent, sanificatori
- CSP: secondo strato, senza 'unsafe-inline'
- CSRF: il browser invia i cookie per conto tuo
- SameSite, token, nuova conferma per le azioni gravi
- Un XSS rompe quasi tutte le altre difese
- Prossima lezione: password, hash e salt

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.